AKV: Agile Read-Efficiently Key-Value OLTP Engine for Non-Volatile Memory

Jianbin Qin, Shuai Liu, Tianyu Wang, *Member, IEEE*, Yuxing Chen, Anqun Pan, Rui Mao, Yuxuan Qiu, Makoto Onizuka, and Chuan Xiao

***Abstract*—Non-volatile memory (NVM), as an emerging storage technology, offers several advantageous features for OLTP engines, including byte-addressability, high capacity, low energy consumption, and data persistence across power failures. Despite these benefits, the current mainstream OLTP engines still commonly adopt a hybrid architecture that deeply couples DRAM with NVM, which results in a complex system architecture and high recovery costs. In this paper, we aim to construct a highly available, stable, and recoverable OLTP engine that guarantees ACID properties through an agile system architecture. We introduce AKV (Agile Key-Value), an NVM-only OLTP storage engine designed to provide effective space utilization, high throughput, and fast failure recovery. AKV addresses the challenges of NVM space management, write redundancy, and concurrency control with two novel techniques: dual-version concurrency control and circular dual-version storage. Experimental results demonstrate that AKV achieves higher throughput (up to 69.7%) and faster recovery (up to 54×) compared to existing storage engines in most scenarios of the TPC-C benchmarks. Additionally, the codebase of AKV (4k+ lines) is more concise than that of SOTA OLTP engines like Zen (8k+ lines) and Falcon (11k+ lines). In addition, this study innovatively proposes a read abort optimization strategy based on dynamic version changes. The experimental results show that this strategy can significantly reduce the transaction abort rate of AKV in specific workload scenarios while maintaining stable system throughput, achieving a maximum reduction of up to 73% in the abort count.**

***Index Terms*—Non-volatile memory, OLTP storage engine, transaction, concurrency control.**

Received 22 March 2025; revised 14 October 2025; accepted 4 November 2025. Date of publication 12 November 2025; date of current version 30 December 2025. This work was supported in part by the National Key R&D program of China under Grant 2022YFC3800602 and Grant NSFC 62472289, in part by Shenzhen Natural Science Foundation under Grant 20220810142731001, and in part by the Guangdong Province Key Laboratory of Popular High Performance Computers under Grant 2017B030314073. Recommended for acceptance by K. Zheng. *(Corresponding authors: Rui Mao; Yuxuan Qiu.)*

Jianbin Qin, Shuai Liu, and Rui Mao are with SICS, Shenzhen University, Shenzhen 518000, China (e-mail: qinjianbin@szu.edu.cn; liushuai2022@email.szu.edu.cn; mao@szu.edu.cn).

Tianyu Wang is with Shenzhen University, Shenzhen 518000, China (e-mail: tywang@szu.edu.cn).

Yuxing Chen and Anqun Pan are with Tencent Inc., Shenzhen 518000, China (e-mail: axingguchen@tencent.com; aaronpan@tencent.com).

Yuxuan Qiu is with the Beijing Institute of Technology, Zhuhai 519000, China (e-mail: qiuyx.cs@gmail.com).

Makoto Onizuka is with Osaka University, Osaka 565-0871, Japan (e-mail: onizuka@ist.osaka-u.ac.jp).

Chuan Xiao is with Osaka University, Osaka 565-0871, Japan, and also with Nagoya University, Nagoya 464-8601, Japan (e-mail: chuanx@ist.osaka-u.ac.jp).

Digital Object Identifier 10.1109/TKDE.2025.3632295

I. Introduction

NON-VOLATILE memory (NVM) is an emerging storage technology that retains data even after a power failure. It has characteristics such as byte-addressability, high capacity, and low energy consumption [1], [2], [3], [4]. NVM is regarded as a disruptive technology capable of revolutionizing computer system architecture. The growing market demand for efficient, fast memory access and reduced power consumption is further driving the advancement of NVM technology.

The storage engine underpins database systems,responsible for managing data, concurrency, and recovery. Modern architectures use DRAM for caching and SSDs for bulk storage, but this introduces challenges like optimizing DRAM caching, SSD write-back strategies, and handling the performance gap between them. These complexities increase uncertainty, energy consumption, and system overhead, especially with larger DRAM setups.

The emergence of NVM offers promising solutions to the aforementioned challenges. In recent years, researchers have explored leveraging NVM in storage engines to enhance performance or reduce costs [5], [6], [7], [8], [9], [10], [11], [12]. For example, redesigned storage engines replace DRAM with NVM as the primary storage [5], [6], [12]; NVCaracal [7] integrates NVM in deterministic databases to support larger datasets at a lower cost per gigabyte and enable faster failure recovery; Other works propose a three-tier buffer manager [8], [9] that uses NVM as an intermediate buffer between DRAM and SSD. Additionally, NVM's byte-addressability enables a more efficient approach to log recovery methods, such as Write-Behind Logging (WBL) [10] and Zen [11], which minimize log records by directly writing updates to NVM.

Most of these storage engines use a hybrid architecture that combines DRAM and NVM. Although this approach has improved system performance to some extent, its highly coupled design paradigm not only significantly increases the complexity of the system architecture, but also leads to higher data recovery costs. For example, it becomes necessary to manage data migration between different memories and ensure data consistency in the event of a DRAM failure. In addition, the presence of multiple copies of data in different storage media can cause write amplification and increase recovery costs. To solve these problems, this work aims to present a novel OLTP storage engine design based on a single-tier NVM architecture to reduce the data management overhead associated with cross-media storage and

simplify system architecture. However, building an NVM-only architecture OLTP storage engine faces three major challenges:

- *NVM Space Management:* OLTP engines typically perform frequent updates. Due to the high cost of NVM persistence operations, current NVM space management strategies can lead to significant overhead.
- *Concurrency Control:* Multi-version concurrency control (MVCC) is the most commonly used method in existing systems. Although MVCC can enhance system performance, it, without optimization, is inefficient for NVM-based OLTP storage engines as it tends to generate long version chains and incur high garbage collection costs.
- *Tuple Storage:* Current OLTP storage engines manage data either through in-place updates combined with logs and checkpoints, or through out-of-place updates with periodic space reclamation. In-place updates, while logged for fault recovery, introduce additional space overhead and may underperform on NVM [10]. Conversely, out-of-place updates can lead to excessive recovery overhead, further degrading overall system performance.

To overcome these challenges, we have designed an OLTP storage engine named *AKV* (Agile Key-Value). First, AKV addresses lengthy version chains through a dual-version concurrency control mechanism, retaining only two versions per tuple (old and new). This minimizes read-write conflicts and reduces the overhead of maintaining version chains in NVM. Second, to achieve lightweight space management and reduce storage overhead, AKV adopts a two-version out-of-place update approach, where updates alternate between the two versions, thereby optimizing NVM space utilization and minimizing space reclamation overhead. Once both versions are created, the tuple's address remains unchanged, which reduces index update frequency. In addition, to reduce transaction abort rates, we propose a dynamic version switching strategy that optimizes the dual version concurrency control mechanism.

We evaluated AKV on YCSB and TPC-C benchmarks. The results show that AKV demonstrates superior throughput across a majority of tested scenarios, compared to traditional storage engines. In particular, AKV not only achieves higher throughput (by 14.5% on YCSB and 69.7% on TPC-C against the runner-up) but also boasts faster failure recovery times (up to 2.1x on YCSB and 54x on TPC-C against the runner-up). Furthermore, AKV achieves these improvements with a more streamlined codebase, containing significantly fewer lines of code (4k+) than other state-of-the-art OLTP engines like Zen (8k+) and Falcon (11k+).

Our contributions can be summarized as follows:

1) We present a novel and agile OLTP storage engine, called AKV, with a compact NVM-only architecture.
2) We propose a *dual-version concurrency control* mechanism in which each tuple retains at most two versions. This design is proven to provide serializable isolation. This approach not only reduces the access cost by multiple tuple versions, but also supports concurrent transactions to read and write tuples simultaneously, leveraging the high bandwidth and low latency characteristics of NVM.
3) We propose a *circular dual-version storage* to handle updates. This method can improve space utilization and read/write performance, while also reducing log write amplification, index modification frequency, and the cost of failure recovery.
4) Furthermore, we introduce a read abort optimization strategy based on dynamic version switching. This strategy significantly reduces transaction abort rates in specific workload scenarios while maintaining stable system throughput.
5) Experimental results on the YCSB and TPC-C benchmarks show that AKV achieves higher throughput than state-of-the-art storage engines in most cases, along with lower abort rates and faster failure recovery.

II. Preliminaries

Byte-addressable NVM is a novel non-volatile memory technology that has attracted the attention of researchers. They mainly focus on the application and optimization of NVM in storage engines, transactional memory systems, fault consistency, and other related areas [7], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19]. This section will first outline the current status and application value of NVM technology, then delves into existing NVM storage engine designs and their limitations.

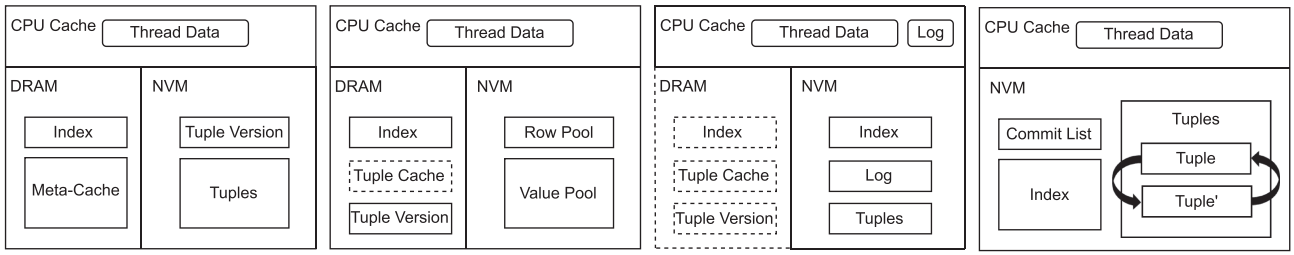
A. Non-Volatile Memory

DRAM stores data using capacitors, but over time the capacitors lose their charge, resulting in data loss. Therefore, regular refreshing is required to ensure data accuracy, which leads to an increase in energy consumption. Prior studies [20], [21] show that DRAM energy consumption accounts for 24% to 40% of the total energy consumption in data centers, with refresh operations accounting for over 40% of this portion. At the same time, it is difficult to increase the storage density of DRAM, and manufacturing chips below 10 nanometers face huge challenges. Therefore, DRAM faces capacity and energy consumption issues, for which NVM presents a potential solution.

NVM stores information by changing the resistance of storage cells, with prominent examples including Phase-Change Memory (PCM) [1], Resistive Random Access Memory (ReRAM) [3], and Spin Orbit Torque Magnetic Random Access Memory (SOT-MRAM) [22]. These technologies support multi-level-cell storage, enhancing storage density and performance. Compared to DRAM, NVM offers advantages such as non-volatility, low energy consumption, and high storage density. Relative to conventional DRAM, NVM can achieve more than ten times the storage capacity, a gap that is expected to continue to widen due to the high storage density characteristics of NVM materials.

B. Existing System Designs for NVM

Existing OLTP engines for NVM are typically adapted from main memory OLTP systems. These systems have to consider concurrency control and crash recovery mechanisms to provide ACID transaction support. NVMs significantly change the way systems store and access persistent data, as applications can directly access persistent data with minimal operating system intervention.



(a) Zen. (b) NVCaracal. (c) Falcon. (d) AKV (ours).

Fig. 1. OLTP engine designs for NVM. The dashed lines represent optional components.

Three-tier Buffer Manager: Renen et al. [8] proposed a three-tier buffer manager that uses NVM as an intermediate layer between DRAM and SSD, storing hot, warm, and cold data in DRAM, NVM, and SSD, respectively. Spitfire [9] improves this design with an adaptive data migration strategy, directly accessing NVM data and reducing duplicate pages between DRAM and NVM, thereby enabling more SSD pages to be cached. However, these managers rely on traditional write-ahead logging (WAL) and checkpointing for fault recovery, which prevents them from fully leveraging NVM's byte-addressability. This leads to redundant writes in NVM, adversely affecting system performance.

Recovery Techniques for NVM: Recovery techniques like Write-Behind Logging (WBL) [10] and Zen [11] utilize NVM's byte-addressability to persist tuple updates directly, thereby minimizing log records. WBL uses group commits with minimal logging (only timestamp pairs per commit) but requires garbage collection of uncommitted changes during recovery. Zen, as shown in Fig. 1(a), stores indexes and metadata in DRAM while keeping tuple versions in NVM. It determines commit status via a tuple header bit, avoiding logging. However, Zen requires multiple database scans and index reconstruction during recovery, with the latter accounting for over 30% of recovery time. Its out-of-place updates also cause frequent index modifications and NVM fragmentation, further increasing system overhead.

NVCaracal: NVCaracal [7], shown in Fig. 1(b), is an NVM-integrated deterministic database that stores indexes and intermediate tuple versions in DRAM, while keys and values reside in NVM, with optional DRAM caching. It mitigates issues in WBL and Zen by keeping at most two tuple versions in fixed NVM positions, thereby reducing index modifications and fragmentation. However, it requires prior knowledge of transaction read-write sets and ordering, which limits its flexibility, and it must log transactions before each epoch, without effectively minimizing logging overhead.

Falcon: Falcon [12], shown in Fig. 1(c), primarily stores indexes, tuple heaps, and logs in NVM, with hot tuples and logs cached in the CPU, and optionally uses DRAM for performance enhancement and multi-version concurrency control. It stores only one tuple version in NVM, reducing fragmentation and index modifications seen in WBL and Zen. However, Falcon relies on the eADR mechanism, and for large tuples, the CPU cache may be insufficient for both logs and hot tuples, leading to frequent log flushes and direct tuple modifications in NVM.

Pisces: Pisces [14] is a memory transaction system that reuses transaction-based redo logs as new data versions. It introduces dual-version concurrency control and a three-stage commit protocol to address the issue of long MVCC version chains, ensuring correctness and improving read performance. However, its dual-version concurrency control can lead to higher transaction abort rates under high write workloads, as it requires frequent space allocation and reclamation, thereby increasing overhead for the space manager. Furthermore, the system supports only snapshot isolation, limiting its applicability to scenarios requiring stricter isolation levels.

AKV: Fig. 1(d) illustrates the design of our storage engine AKV, in which the main components, including indexes, tuple heaps, and commit lists, reside on NVM. The main difference from Zen is that we retain only up to two versions of tuples in the tuple heap and utilize persistent indexes stored in NVM. Compared to NVCaracal, AKV supports real-time concurrency control without requiring prior knowledge of transaction read and write sets, writes fewer logs to NVM, and employs NVM for indexing. Overall, our design is more general and generates fewer logs than NVCaracal and Falcon, while addressing the drawbacks of WBL and Zen mentioned earlier. Moreover, AKV facilitates faster fault recovery. Finally, compared to Pisces, we have optimized space allocation and provide serializable isolation level support.

III. System Design

Our objective is to simplify the system architecture by leveraging the characteristics of NVM, including reducing logging and space reclamation overhead, while ensuring a certain level of recovery speed and system performance. Therefore, we have identified three design considerations for applying NVM in OLTP storage engines: (1) **Simplicity:** Adopting an NVM-Only architecture to reduce the data management overhead associated with hybrid storage media; (2) **Efficiency:** Redesigning concurrency control to take advantage of NVM's asymmetric read/write performance, specifically its faster read speed; (3) **Space optimization:** Utilizing the byte-addressability of NVM to facilitate tuple data and space reuse.

A. NVM-Only Architecture

We explore an NVM-only storage engine architecture, which places all components (such as indexes, tuples, directories, logs,

TABLE I
COMPARISON OF DIFFERENT STORAGE MEDIUM [24]

	DRAM	PCM	FRAM	SOT-MRAM	ReRAM
Read Time	<10ns	48ns	120ns	1.1ns	5ns
Write Time	<10ns	150ns	120ns	1.4ns	10us
Read Energy	Very Low	Very Low	Very High	High	Very High
Write Energy	Very Low	Very Low	Very High	High	Very Low
Static Energy	High	Low	Low	Low	Low

space managers, etc.) in NVM without using DRAM cache. Unlike DRAM+NVM systems that face data migration, consistency, and redundancy challenges, the NVM-only approach is simpler, as it writes data directly to NVM. Benefiting from byte-addressability, non-volatility, and higher density than DRAM, NVM enables servers to host larger memory capacities within limited DIMM slots. This architecture reduces complexity and failure recovery costs, offering a streamlined alternative to traditional designs. All components are deployed on NVM, enabling the system to recover rapidly from failures [23], thus minimizing the impact of database interruption on business continuity.

Based on the research and analysis of the relevant literature [24], [25], [26], [27] and commercially available memory technologies, we summarize the key characteristics in Table I. It provides an overview of the properties of different memory types. DRAM typically requires self-refreshing to prevent data loss, which incurs high static energy consumption. In contrast, NVM does not require refreshing to preserve data, resulting in very low static energy consumption [28]. As the DRAM capacity of the server increases, the energy consumption will also rise. Meanwhile, NVM technologies such as SOT-MRAM offer read and write speeds comparable to DRAM. By storing all database components in NVM and using DRAM only for initializing processes and other essential steps, the system minimizes its DRAM dependency. This approach can reduce system energy consumption and provide a larger storage capacity with near-DRAM speed.

B. Dual-Version Concurrency Control

Concurrency control manages the way transactions are executed and coordinated. Common techniques include those based on two-phase locking, timestamp ordering, optimistic concurrency control (OCC), and MVCC [29], [30], [31], [32], [33], [34]. DBx1000 [31] and Zen [11] implement MVCC combined with timestamp ordering, while Falcon [12] combines MVCC with OCC. We conducted YCSB tests on these three systems using multi-version and single-version concurrency control, and found that MVCC did not consistently improve system performance; in some cases, it even resulted in throughput degradation. Moreover, maintaining excessive data versions in memory can significantly exacerbate the complexity of version chain management and may lead to memory fragmentation and resource exhaustion.

Pisces [14] and MCSI [35] offer valuable insights for concurrency control on NVM. We re-examine MVCC and adopt a dual-version concurrency control (DVCC) approach, which ensures that each tuple has a maximum of two versions: 1) the new (modifiable) version and the old (read-only) version; 2) both versions are readable, with transactions selecting the appropriate one based on their start timestamp. DVCC offers faster read performance while leveraging the old version as an implicit log to facilitate undo operations for uncommitted transactions during recovery or rollback, thereby reducing logging overhead. Pisces combines DVCC with a three-phase commit protocol to achieve snapshot isolation. However, as storage engines often require serializable isolation levels, necessitating an additional validation process is necessary to ensure correctness when using DVCC.

C. Circular Dual-Version Storage

Compared to MVCC, DVCC can lead to higher transaction abort rates. Furthermore, a simple out-of-place storage strategy can lead to excessive unavailable space allocations, increasing space management overhead. To address these issues, we adopt a circular dual-version storage approach, where updates are cyclically applied to two fixed versions without requiring additional space reclamation. This significantly reduces space management costs while maintaining high performance. Previous research on in-memory OLTP systems has demonstrated that similar circular versioning methods exhibits excellent performance [36], [37]. Moreover, due to NVM's byte-addressable nature, which is comparable to DRAM, this approach is equally well-suited for NVM-based storage systems. This design offers several benefits: Index modifications are avoided if the storage locations of the versions remain unchanged, transaction aborts are efficiently handled by resetting the updated tuples, and space reclamation is generally unnecessary, thereby simplifying space management.

This optimization works well for fixed-length records. However, for variable-length records, we need to ensure that updates fit within the allocated space for two versions. To address this, we introduce a mechanism that checks whether an update fits within the pre-allocated space. If it does not, new storage is allocated, the two versions are re-linked, and the old version's space is reclaimed. If the update affects the index, the index is also updated accordingly. This approach introduces space management challenges due to varying reclaimed space sizes. To reduce fragmentation and improve space utilization, the reclaimed space is sorted by size, and a best-fit allocation strategy is employed. This ensures efficient space reuse while maintaining circular dual-version storage benefits. For simplicity, we focus on fixed-length records but provide a configurable option for variable-length records.

IV. IMPLEMENTATION

In this section, we present the concrete implementation of our storage engine AKV based on the aforementioned design principles. The discussion encompasses the system's overall architecture, tuple metadata design, and transaction processing workflow. We elaborate on the concurrency control algorithms with formal proofs, followed by an extension of dynamic versioning for concurrent control. Additionally, we detail the mechanisms for transaction abort handling, garbage collection, as well as logging and recovery processes.

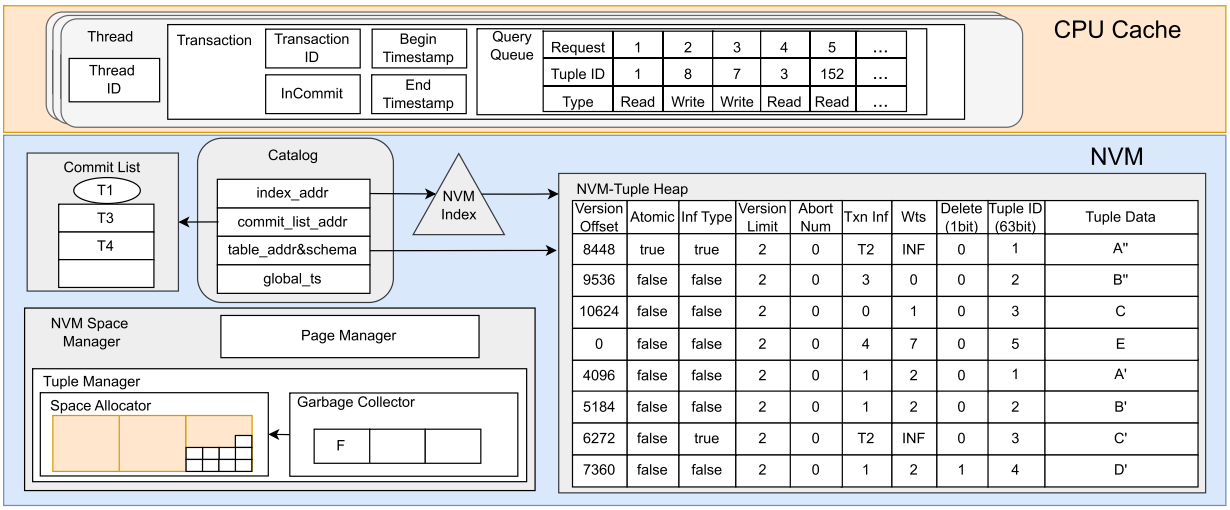


Fig. 2. AKV architecture.

A. Architecture Overview

As shown in Fig. 2, the system architecture includes the following core components: the index, tuple heap, space manager, directory, and commit queue. In this study, we designed the AKV system based on an NVM-only architecture, where all core components (including database tuples, metadata for concurrency control, and recovery structures) are stored in NVM, and the storage engine directly performs tuple modification operations on NVM. Due to the high decoupling of certain components within the storage engine, some of them can be migrated to DRAM to achieve higher performance, although this requires specific trade-offs in the architectural design. In subsequent experiments, we conducted detailed testing and evaluation of the approach of deploying some components in DRAM.

NVM-Tuple Heap: NVM-tuple is a persistent tuple in NVM, with all tuples of each table stored in the corresponding NVM tuple heap. These tuples are modified directly on NVM, eliminating the need to cache them in DRAM. In the NVM tuple heap, a logical tuple typically maintains only two versions. The header of an NVM tuple includes the tuple ID (Delete and Tuple ID, 8 bytes), write timestamp (Wts, 8 bytes), transaction modification information (Txn Inf and Inf Type, 8 bytes and 1 B), offsets to other versions (Version Offset, 8 bytes), an atomic boolean for locking(Atomic, 1 B), the tuple version limit (Version Limit, 2 bytes) and the number of transaction read aborts (Abort Num, 4 bytes) caused by the current tuple. The highest bit of the tuple ID is used as a deletion mark. The write timestamp and transaction information (transaction ID or pointer) are used for concurrency control and fault recovery. The transaction pointer helps determine if a transaction is in the commit phase, while the transaction number is used for fault recovery. Offsets are used to access other versions of the tuple, ensuring persistent access to these versions after a system failure.The tuple version limit, combined with the number of transaction read aborts caused by the current tuple, is used to dynamically adjust the number of versions of the tuple.

NVM Index: In NVM, we maintain indexes for each table to leverage its high-speed read and byte-level addressing capabilities, thereby enhancing data access efficiency. AKV employs a design where indexes and data are stored separately in different regions of NVM. Given that our research focus is not on indexing and that OLTP workloads typically involve small-range updates and queries, we adopt a commonly used hash index as a representative example. Within the AKV architecture, other types of indexes can also be seamlessly integrated.

Commit List and Catalog: The commit list is a lightweight log used to record the transaction IDs of committed transactions and to set checkpoints. After a transaction commits, its transaction ID is stored in the commit list. The catalog stores the database's metadata, including addresses for indexes, tuple heaps, commit lists, and table schemas, among other information. During system recovery, the catalog is accessed first. The system then reconstructs the NVM address mapping based on the table schemas and index information in the catalog and updates these addresses back into the catalog.

NVM Space Management: The NVM space manager in AKV is responsible for the allocation and reclamation of NVM-tuple space. Similar to previous systems [11], [12], AKV employs a two-level management scheme: a page manager and a tuple manager. Each tuple is fixed in length by default. If variable-length records are supported and the record length exceeds the fixed size, the tuple manager allocates a contiguous space of appropriate size by combining tuples within the corresponding page.

B. Transaction Processing

Fig. 3 demonstrates the transaction execution process. At the start of a transaction, the system assigns a start timestamp (①). During the execution phase, the transaction locates the tuple address through an index (②), and directly reads or updates data from the NVM tuple heap (③), or inserts new tuples, allocating space from the tuple-level space manager (④ and ⑤).

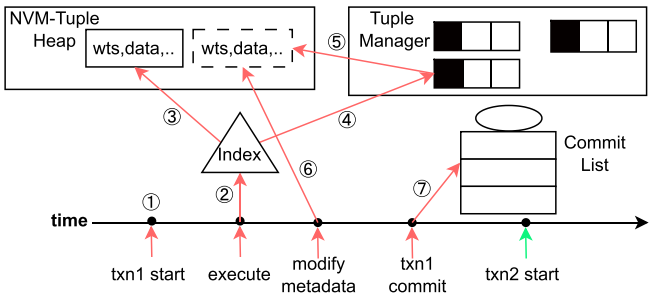


Fig. 3. Lifetime of transactions.

Algorithm 1: Write Operation.

```
1  Function txnWrite(key, txn):
2      tup1 = offsetToPtr(readIndex(key));
3      if tup1 == NULL ∧ txn.type == insert then
4          tup1 = allocator();
5          memcpy_content(tup1, txn);
6          return insertIndex(tup1) ? tup1 : Abort;
7      else if tup1 == NULL then
8          return Abort;
9      tup2 = offsetToPtr(tup1.next);
10     if tup2 == NULL then
11         tup2 = allocator();
12         if CAS(tup1.lock, false, ture) fails then
13             return Abort;
14         bind(tup1,tup2);
15         memcpy_content(tup2, txn);
16         change(tup1.lock);
17         return tup2;
18     if (tup1.wts > txn.ts) ∨ (tup2.wts > txn.ts) then
19         return Abort;
20     else
21         tup = tup1.wts < tup2.wts ? tup1 : tup2;
22         if CAS(tup.lock, false, ture) fails then
23             return Abort;
24         memcpy_content(tup, txn);
25         change(tup.lock);
26         return tup;
```

After completing all operations, the transaction acquires an end timestamp, sets its InCommit status to true, and checks whether any read tuples have been modified by other transactions during execution. Subsequently, it updates the tuple metadata, setting the transaction's end timestamp as the tuple's write timestamp (⑥), ensuring the creation of the transaction version. Upon completion, the transaction ID is recorded in the commit list (⑦), marking the end of the transaction. If a system crash occurs, upon restart, the space allocated to unfinished transactions will be reclaimed, and the tuple version being modified will be marked as unavailable.

Algorithm 2: Read Operation.

```
1  Function txnRead(key, txn):
2      tup1 = offsetToPtr(readIndex(key));
3      if tup1 == NULL then
4          return Abort;
5      fastReadOrWait();
6      tup2 = offsetToPtr(tup1.next);
7      if tup2 == NULL then
8          return (tup1.wts > txn.ts) ? Abort : tup1;
9      fastReadOrWait();
10     tup_old = (tup1.wts < tup2.wts) ? tup1 : tup2;
11     tup_new = (tup1.wts < tup2.wts) ? tup2 : tup1;
       // 0 indicates an invalid tuple
12     if tup_old.wts == 0 then
13         return (tup_new.wts > txn.ts) ? Abort :
               tup_new;
14     if tup_old.wts > txn.ts then
15         return Abort;
16     else if tup_new.wts > txn.ts then
17         return tup_old;
18     else
19         return tup_new;
```

Algorithm Description: Algorithm 1 describes the write operation under dual-version concurrency control. The process begins by searching for the tuple key in the index. If the key does not exist, the system determines whether to insert a new tuple or abort the transaction based on the request type, while handling index-level concurrency to prevent overwrite conflicts from concurrent transactions attempting to insert the same key (Lines 2–8). If the tuple contains only one version, the transaction creates a new version (Lines 10–17). When two versions already exist, the system checks the commit timestamps of both versions. If the commit timestamp of either version exceeds the transaction's start timestamp, the transaction is aborted. Otherwise, the write operation is performed on the older version (Lines 18–26). Before any modification, the tuple must be locked; to minimize locking overhead and improve efficiency, the system employs Compare-And-Swap (CAS) operations. We abstract the insert, update, and delete operations on tuples as write operations. Among these, insert and update operations are processed directly according to the algorithm's logic. For delete operations, we implement them by updating the deletion flag of the tuple, rather than physically deleting the tuple immediately. This design allows subsequent transactions to recognize that the tuple has been deleted, while retaining the old version to ensure that older transactions can still access the required data version.

Algorithm 2 describes the read operation under dual-version concurrency control. The operation begins by searching for the tuple key in the index. If the key is not found, the transaction is aborted (Lines 2–4). Before accessing the tuple, the system determines whether to proceed immediately or wait for the committing transaction that modifies the tuple, based on the

Algorithm 3: Transaction Commit.

```
1  Function txnCommit(txn,readset):
2      set txn.end_ts;
3      if !read_set.empty() then
4          for i = 1 to read_set.size do
5              if hasChanged(read_set[i-1]) then
6                  return Abort;
7      if !write_set.empty() then
8          for i = 1 to write_set.size do
9              changeMetadata(write_set[i-1]);
10     add txn.id to commit list;
11     return Commit;
```

transaction's state (Lines 5, 9). If a version of the tuple exists and the transaction's timestamp is greater than the write timestamp of the tuple, the transaction can read the tuple; otherwise, it aborts (Lines 7-8). To intuitively demonstrate this, we compare two versions of the tuple here and use new variables to reference the new and old versions of the tuple (Lines 10-11). We use 0 to indicate an unavailable tuple version. If the old version is unavailable, only the new version is considered. If the transaction's start timestamp is greater than the new version's timestamp, it can read it; otherwise, it aborts (Lines 12-13). If both versions are valid, the decision on which version to read is based on the comparison between the transaction's start timestamp and the write timestamps of the two versions (Lines 14-19).

Algorithm 3 describes the transaction commit process. The process begins by assigning the transaction's end timestamp (Line 2). Next, the algorithm verifies the tuples in the read set to check if any have been modified by other transactions during the execution of the current transaction. This is achieved by comparing the transaction's start timestamp with the write timestamps of the tuples. If a tuple has been modified, (i.e., its latest write timestamp is greater than the transaction's start timestamp), it indicates the transaction may have read a stale version. To ensure serializable isolation, transactions encountering such conflicts must abort (Lines 3-6). Subsequently, the algorithm updates the metadata of the tuples in the write set to reflect the transaction's changes (Lines 7-9). Finally, the transaction ID is added to the commit queue to finalize the commit process (Line 10).

C. Proof of Correctness

We will first present the proof process for snapshot read-write operations, following with a proof of serializable snapshot isolation.

Theorem 1 (SNAPSHOT WRITE): If two transactions update the same tuple, then one transaction's start timestamp should be greater than another's end timestamp.

Proof: Before attempting to modify the tuple metadata, we first acquire a lock on the tuple (Algorithm 1, lines 12 and 22), and release this lock after the metadata modification is complete. This mechanism ensures that at any given moment, only one transaction can modify the metadata of the tuple. The metadata of the tuple includes a timestamp of the current version, which is set to infinity until the transaction commits. This setting can be used to quickly abort conflicting transactions (Algorithm 1, line 18). Therefore, we only need to prove that for two transactions T_i and T_j updating the same tuple x (where transaction T_j starts before T_i), their end timestamp and start timestamp satisfy the following condition: $T^j_{end_{TS}} < T^i_{start_{TS}}$. Assuming $T^j_{start_{TS}} < T^i_{start_{TS}} < T^j_{end_{TS}}$, we can consider the following two scenarios: (i) T_j updates the tuple x before T_i. When T_i attempts to update x, since T_i's start timestamp is less than the current tuple's write timestamp, T_i will be aborted. (ii) T_i updates the tuple x before T_j. When T_j attempts to update x, T_j will be aborted due to a lock conflict or an unavailable timestamp check. □

Theorem 2 (SNAPSHOT READ): If a transaction Tr reads a tuple x with timestamp TS_x, then: 1) Tr's start TS is not less than TS_x; and 2) There does not exist a transaction Tw that updates x and its end TS is larger than TS_x, but not greater than Tr's start TS.

Proof: The first condition ensures that the transaction's start timestamp is larger than the tuple timestamp by comparing the timestamps of different versions of the tuple (Algorithm 2, lines 8, 12-19). For the second condition, which needs to be discussed separately, the transaction will be in two states when modifying the tuple, the run state and the commit state. While in the running state, the end timestamp of the transaction is not determined, and Tr reads the tuple modified by Tw and does not access the version it created. When in the commit phase, at which point Tw has already assigned an end timestamp, Tr needs to wait for Tw to commit (as shown by the **incommit** in Fig. 2) to access the tuple modified by Tw before determining the version to access. This ensures that the tuple version Tr created by Tw is always visible if the end timestamp of Tw is less than the start timestamp of Tr. □

Theorem 3 (SERIALIZABLE SNAPSHOT ISOLATION): Transactions submitted are sorted based on their end timestamps, and transactions that were not successfully submitted are deleted.

Proof: To extend the proof to serializable snapshot isolation, we must further ensure that the read set of each transaction does not exhibit read-write conflicts with concurrently committed transactions. After obtaining the transaction's end timestamp, we determine whether the tuples in its read set have been modified and whether the modified versions have been committed. If this tuple is modified with an end timestamp larger than the current transaction start timestamp but smaller than the end timestamp, the current transaction is aborted to ensure that the transactions are serialized in the order of the end timestamp. This is because there exists a committed transaction that modified the data item after the current transaction's read operation and before the current transaction's commit, which violates the requirements for serializability. □

D. Dynamic Version Number

The dual-version concurrency control mechanism, by limiting the number of versions per tuple, can lead to a significant

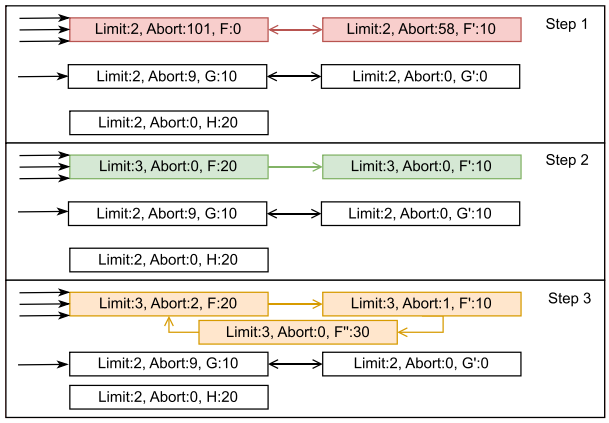


Fig. 4. Detailed process of version dynamic transformation.

increase in transaction abort rate in high concurrency scenarios, thereby affecting the overall performance of the storage engine. Although the previously proposed circular dual-version storage mechanism effectively reduces abort overhead, it does not fundamentally reduce the frequency of transaction aborts. In the dual-version concurrency control we designed, the abort conditions for write operations are relatively strict, while the abort conditions for read operations are relatively relaxed, only aborting when a transaction cannot read a tuple version earlier than its start timestamp. However, due to the limitation of dual-version, even in cases where the write operation ratio is low, it may still cause a large number of transaction aborts. To address this issue, we have introduced a dynamic version control strategy. This strategy defaults to maintaining dual-version, but dynamically increases the number of versions for tuples that frequently abort due to read conflicts.

Fig. 4 illustrates the dynamic version expansion process controlled by two metadata fields: version limit (Limit) and abort count (Abort). When concurrent access to tuple F causes its abort count to exceed a predefined threshold (e.g., 100), the system triggers version expansion upon the next update. This process involves incrementing the version limit while resetting the abort count, disconnecting the existing version ring, updating corresponding index entries, and finally incorporating the new version into a reconstructed version chain. Through this mechanism, AKV achieves adaptive version management that dynamically adjusts to workload contention levels.

The overall mechanism builds upon dual-version concurrency control and cyclic dual version storage, but enhances them by allowing each tuple to flexibly adjust the number of versions and introducing additional checks during version comparison, it ensures that read and write operations can correctly access the appropriate version.This improvement is mainly reflected in the logical changes in the 18th line of Algorithm 1 and the 10th to 19th lines of Algorithm 2.

E. Logging and Durability

AKV uses lightweight logging approach that records only transaction IDs to guarantee crash consistency. Since the log is written after data modification, transactions recorded in the commit list after a crash are guaranteed to be committed. Although previous tuple versions must be retained, they are necessary to support read-write concurrency. A checkpoint mechanism is still essential, as the continuously growing commit list would otherwise impair recovery. When the number of transaction IDs reaches a threshold, the commit thread determines the maximum committed transaction ID and identifies the uncommitted IDs between this maximum and the previous checkpoint, recording them as a new checkpoint. Additional periodic checkpoints further enhance reliability and recovery efficiency.

Recovery primarily involves tuple verification, index maintenance, and restoration of auxiliary structures. For tuples, the commit list and checkpoints are used to identify and invalidate modifications by uncommitted transactions, ensuring data consistency. For indexes, which adopt a hash structure, recovery is generally unnecessary except for handling insertions and deletions: for inserted tuples, the system verifies that index entries point to the current version; for deletions, it relies on tuple-level deletion markers. Auxiliary structures, including the directory, commit list, and space manager, are persisted on NVM. Directory updates and commit list appends follow a post-write strategy that may introduce redundancy but guarantees durability. During recovery, their current states are directly loaded, while the space manager ensures correct allocation and recycling of storage through tuple or page metadata.

V. Evaluation

In this section, we will conduct experiments on machines equipped with PMem, and compare the performance with several state-of-the-art OLTP storage engines.

A. Experimental Setup

Configurations: The experimental equipment is configured with 2 Intel Xeon Gold 6326 CPU with 16 cores/32 threads, 512 KB L1I, 718 KB L1D, and 20 MB L2 cache, along with a shared 24 MB L3 cache. The system contains 256 GB (8x32 GB) of DRAM and 2 TB (8x256 GB) of Intel Optane DC Persistent Memory (PMem) based on 3DXPoint. We configured PMem in App-Direct mode, allowing NVM to be mapped to the software's virtual addresses. The experimental environment is Ubuntu 20.04.6 LTS and 5.4.0-169 generic Linux kernel. We mounted the file system onto NVM in fs-dax mode, then used libPMem in PMDK [38] to map the NVM files to the virtual memory of the process. All codes are written in C/C++ and compiled using GCC version 9.4.0. We conduct experiments on a single CPU slot, utilizing its associated NVM and DRAM. A single CPU slot supports up to 1 TB of NVM and 128 GB of DRAM.

Benchmarks: We evaluate AKV using both uncontended and contended workloads in two benchmarks: YCSB [39] and TPC-C [40]. Each thread executes 100000 transactions, during which if a transaction is terminated due to a conflict, the thread will redo the transaction.

YCSB is a widely used key-value workload that represents the transactions handled by internet companies. In our experiments, the YCSB database consists of a single table. Each tuple contains an 8-byte primary key and ten data columns of random string data, each with 100 bytes. The size of a tuple is approximately

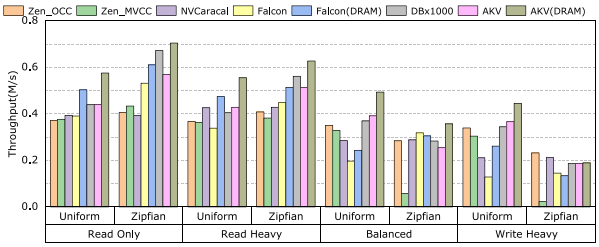


Fig. 5. YCSB throughput (16 threads).

1 KB. By default, each YCSB transaction is composed of 16 random requests. Given a primary key, read requests retrieve the tuple, and write requests modify the tuple. We refer to Zen's experimental setup and vary two parameters in the workload: (1) the percentage of read/write requests, including Read-Only (RO, 100% read), Read-Heavy (RH, 90% read, 10% write), Balanced (BA, 50% read, 50% write), and Write-Heavy (WH, 10% read, 90% write); (2) the root mean square parameter of the Zipfian distribution: Uni. ($\theta = 0$) and Zipf. ($\theta = 0.9$). Here, Uni. has no request locality, while Zipf. is used to simulate high transaction conflict situations. We default to using a database of one million tuples($\approx$ 100GB) for YCSB.

The TPC-C benchmark test is an important tool for evaluating the performance of OLTP systems. It simulates the business activities of a wholesale supplier by designing 9 tables and 5 transaction types for a comprehensive performance assessment. Among them, the New-Order and Payment transaction types are the most common, accounting for 45% and 43% of all transactions, respectively. These two short read-write transactions together constitute 88% of the TPC-C workload. Therefore, we focus on these two transactions. TPC-C allows users to adjust the number of warehouses to simulate different contention levels, thereby assessing the system's performance under different loads and pressures. We used different configurations of TPC-C to evaluate the system's performance in various contention scenarios: 256 warehouses for low contention scenarios, 8 warehouses for medium contention scenarios, and a single warehouse for high contention scenarios.

Compare with other OLTP engines: After an in-depth investigation into the latest research on NVM-based OLTP storage engines, we have chosen Zen, NVCaracal, and Falcon as our experimental comparison subjects. In addition, we selected the DBx1000 with logging enabled as a comparison reference for the memory storage engine. Zen is an outstanding work in the DRAM+NVM architecture, utilizing the out-of-place update to modify tuples, which has certain similarities with our dual-version storage. Zen's open-source code provides options for concurrency control, so we selected commonly used concurrency control methods, OCC (optimistic concurrency control), and MVCC for comparison. However, since Zen's source code does not support TPC-C tests, we did not include Zen in the TPC-C comparison experiment. Also, we compare our dual-version storage mechanism with NVCaracal's dual-version checkpoint design. Since NVCaracal currently only supports configuring the number of threads in multiples of eight, and its maximum configurable thread count is limited by the number of CPU cores, the available sample points are relatively few. Therefore, NVCaracal was not included in the scalability experiments. Falcon can be set to an NVM-only architecture, making it a good reference. However, since we did not consider the eADR mechanism, we excluded Falcon from the recovery experiment. Since DBx1000 with logging enabled only supports no-wait concurrency control based on timestamp sorting in its open source implementation, we will only use it for comparative experiments on transaction throughput.

B. Transaction Performance

YCSB Performance:

We evaluate the throughput of different systems by varying request proportions and distribution, in the YCSB benchmark test. As shown in Fig. 5, for all the target storage engines, we have made two observations: (1) As the write ratio increases, the transaction throughput shows a downward trend, mainly due to the increased number of writes to NVM. Note that, the read speed is typically faster than the write speed in NVM. Meanwhile, due to the impact of concurrency control, an increased write ratio usually leads to a higher number of transaction aborts. (2) As the transaction conflict increases, transaction throughput is also affected. In read-heavy situations, high transaction conflicts lead to increased transaction throughput. This is because more intensive transaction access causes the CPU cache to store data that is likely to be accessed soon, thereby enhancing throughput. However, in write-heavy situations, high transaction conflicts lead to decreased transaction throughput. This is due to the increase in write conflicts causing more transaction rollbacks, which in turn reduces throughput.

For Zen, Zen_MVCC has a significant drop in throughput compared to Zen_OCC when the write ratio is high, especially for the Zipf. distribution. By investigating the source code of Zen_MVCC, we found two main reasons for the low throughput: (1) To ensure transaction correctness, subsequent read and write operations have to wait until conflicting transactions' write operations are completed. (2) As the number of write versions increases, not only does the overhead of garbage collection rise, but the read operations also take more time to identify the appropriate tuple version due to the excessive versions. For the Uni. distribution, the impact of MVCC is not significant due to fewer conflicting operations among transactions. However, for Zipf. distribution, the increase in conflicting operations makes the above two situations more pronounced. For NVCaracal, since it does not support read-only transactions, we made minor modifications to its code and conducted a fair comparison. We measured the time overhead of NVCaracal and found that the time spent in the initialization phase is comparable to that in the execution phase. This is because NVCaracal is a deterministic database that pre-serializes transactions before execution and then executes them in the pre-assigned order, with no transaction abort.

For DBx1000 with logs, because it is a in-memory database, it can show high transaction throughput in most cases. However, its concurrency control mechanism is limited to the no-wait timestamp-based sorting strategy, and requires additional disk

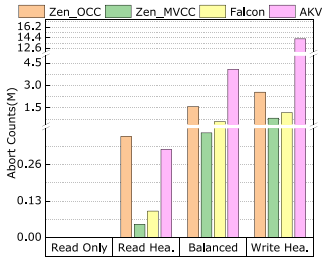


Fig. 6. YCSB abort counts.

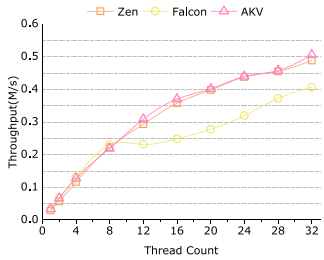


Fig. 8. YCSB scalability.

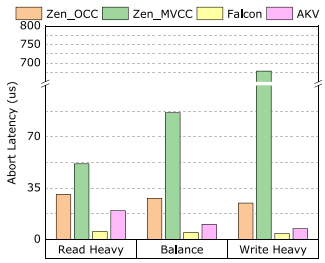


Fig. 7. YCSB abort latency.

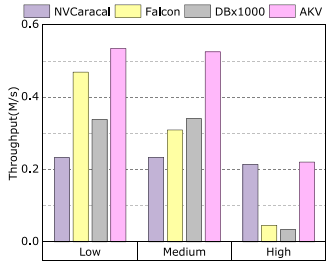


Fig. 9. TPC-C throughput.

flushing operations to persist logs. Therefore, its performance in write intensive scenarios is relatively general.

We tested the solely NVM version of Falcon and its version with indexes stored in DRAM. The results indicate that, in most cases, the Falcon version with DRAM indexes outperforms Zen and NVCaracal. The reason is that it employs an in-place update mechanism and only persists the final results of transactions to NVM, while intermediate results and logs are efficiently processed in the CPU cache. This strategy not only reduces the frequent updates of indexes but also significantly decreases the need for data migration between different storage mediums. We consider using DRAM as an optional option to achieve more efficient data access by migrating some components to DRAM. So there are two versions of AKV in the experiment: NVM-only and NVM + DRAM index. The results show that, in most cases, AKV deployed entirely on NVM achieves higher throughput than other storage engines that also leverage NVM, and its performance approaches that of DBx1000 with logging. One reason is that the dual-version concurrency control adopted by AKV ensures serializable snapshot isolation level while maximizing system read performance. Therefore, AKV performs the best in read-heavy situations. In Fig. 6, it is shown that AKV generates a large number of transaction rollbacks under the write-heavy situation. At this point, the circular dual-version storage we adopt reduces the cost of transaction rollback by fixing the storage location of tuples. As shown in Fig. 7, the rollback delay of AKV is less than one-third of Zen_OCC. Falcon stores transaction information in the CPU cache and writes it to NVM only upon commit, thereby incurring very low rollback overhead. This design also explains why Falcon achieves higher throughput under workloads with high Zipf skew. In the case of write heavy and Zipfian, MVCC generates about 20.3 M extra tuples of space overhead (approximately 20 GB), while DVCC generates about 9.0 M extra overhead (approximately 9 GB). When the initial number of tuples is expanded to 10 M and other conditions remain unchanged, MVCC's additional overhead further increases to 20.5 M (approximately 20 GB), while DVCC's is 7.4 M (approximately 7 GB). These results show that in most scenarios, cyclic dual version storage can significantly alleviate the space overhead brought by traditional MVCC. Similar to Falcon, by fixing the position of tuples and directly accessing data on NVM, AKV reduces the overhead of index updation and data replication between different storage mediums. Without considering DRAM indexes, AKV achieves performance improvements of 12.5% and 14.5% in read-only and read-heavy respectively, while ensuring that data is not lost in case of power failure. The experimental results show that using DRAM indexing can improve system performance, but it also brings about an increase in system architecture complexity and DRAM resource utilization. Therefore, it is recommended to select targeted solutions based on the performance requirements and resource constraints of actual application scenarios.

YCSB Scalability: We have studied the scalability of Zen, Falcon and AKV. We use Balanced, Uniform requests in the experiments. As shown in Fig. 8,all three storage engines exhibit good scalability. However, it can be observed that the throughput does not increase linearly with the number of threads; instead, the performance gain gradually diminishes as concurrency grows. This trend is primarily attributed to the latency incurred by cross-node data transfers in the NUMA architecture and the higher likelihood of transaction conflicts under high contention.

TPC-C Performance: Fig. 9 shows the transaction throughput of Falcon, NVCaracal, DBx1000 with logging and AKV under the TPC-C workload, using 16 threads. Among them, Falcon uses OCC as the concurrency control strategy and only uses NVM, without enabling DRAM to store data. In general, AKV

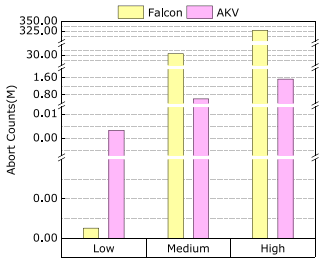


Fig. 10. TPC-C aborts counts.

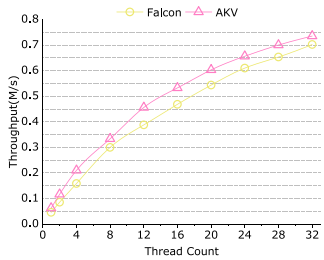


Fig. 11. TPC-C scalability.

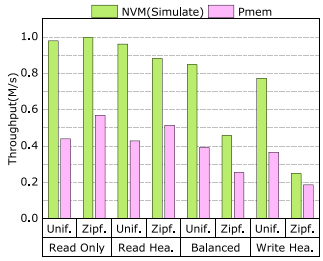


Fig. 12. YCSB throughput with simulated NVM.

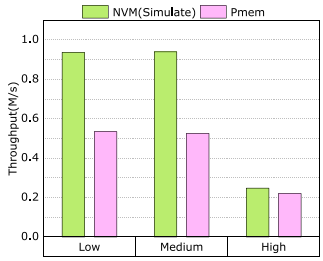


Fig. 13. TPC-C throughput with simulated NVM.

performs the best. Under low, medium and high contention loads, the throughput of AKV is 13%, 54% and 3% higher than that of the best scheme among the other three. In the case of medium contention, Falcon's throughput begins to decline, while AKV and DBx1000, NVCaracal's throughput remain relatively stable. From Fig. 10, it can be seen that the performance degradation of Falcon is mainly due to a large number of transaction abort. Although AKV has an increase in the number of interruptions compared to low contention, its impact on throughput is not significant due to its lower interruption delay and much fewer interruptions compared to Falcon. Under the high contention load, the throughput of AKV is significantly higher than that of Falcon and DBx1000, but basically the same as that of NVCaracal. From Fig. 10, it can be seen that the number of abort for both Falcon and AKV has significantly increased, resulting in a continued decrease in their throughput. Because DBx1000 adopts the no-wait concurrency control strategy, it also leads to a high transaction abort overhead. It's worth noting that NVCaracal's performance is almost the same in both situations. This is mainly because it has determined the execution order of transactions at the initialization stage, so it will not produce performance fluctuations due to changes in contention.

TPC-C Scalability: We tested the TPC-C scalability of AKV using Low contention scenarios and Falcon as a comparative reference. As shown in Fig. 11, the experimental results demonstrate that AKV exhibits good scalability. However, the throughput does not increase linearly with the number of threads; instead, the performance improvement gradually diminishes as concurrency grows, which is consistent with the trend observed in the YCSB benchmark. This phenomenon can be mainly attributed to the latency introduced by cross-node data transfers in the NUMA architecture and the increased likelihood of transaction conflicts under high contention.

C. Simulated NVM

To demonstrate that our design can run on different NVM devices and that performance will correspondingly improve with increased NVM bandwidth and reduced latency, we conducted experiments simulating NVM. We conduct experiments using DRAM-emulated PMem to evaluate system generality. While emulated PMem exhibits lower latency and higher bandwidth than PMem, these characteristics represent an idealized NVM closer to DRAM, providing valuable insights into future NVM architectures.

Our evaluation using the YCSB benchmark on AKV shows significant performance improvements with DRAM-emulated NVM compared to PMem(Fig. 12). This enhancement stems from the emulated environment's superior characteristics: reduced access latency, increased bandwidth, and true byte-addressability properties that approximate an optimal NVM implementation. Similarly, TPC-C experiments demonstrate substantial performance gains with DRAM-emulated NVM (Fig. 13). These results strongly indicate that as NVM technologies evolve to bridge the performance gap with DRAM, our storage engine architecture is well-positioned to deliver optimal performance.

D. Impact of Versions and Space Allocation

In this section, we discuss the impact of version number and space allocation on AKV's concurrency control. Fig. 14 illustrates the performance implications of different version control mechanisms: Dynamic represents our adaptive version control with the abort threshold set to 1024. This value is the optimal value determined after weighing the space occupation and transaction throughput through multiple rounds of testing. DVCC denotes our dual-version concurrency control TVCC restricts the maximum versions to three; QVCC limits the maximum

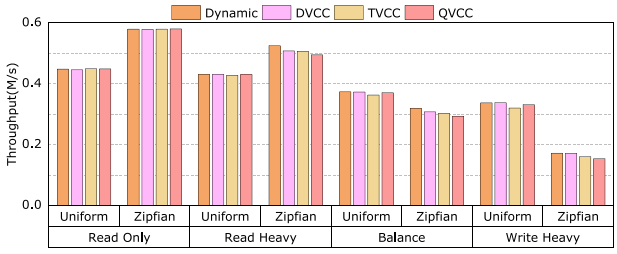


Fig. 14. Impact of versions.

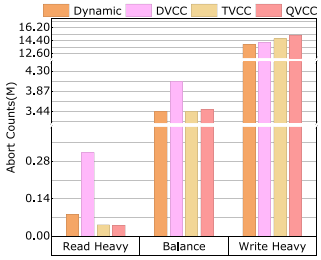


Fig. 15. Abort counts.

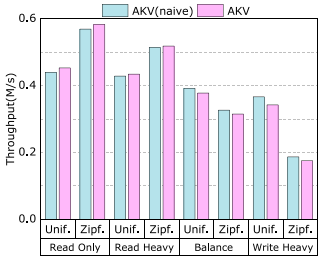


Fig. 16. Impact of allocation.

versions to four. Fig. 15 presents the abort counts across different concurrency control schemes, excluding read-only and uniform scenarios as they exhibit negligible abort rates. Our analysis reveals that the dynamic version control mechanism achieves superior throughput across most scenarios. This performance advantage stems from its ability to maintain the dual-version control under low read-abort conditions, thereby minimizing rollback and read overheads, while dynamically increasing version numbers to enhance read concurrency and reduce abort rates when encountering frequent read conflicts. When the reading ratio is high, it can reduce the number of interruptions by 73%. Notably, our experimental results demonstrate that the dual-version concurrency control often outperforms both three-version and four-version implementations in most operational scenarios.

Fig. 16 illustrates the transaction throughput differences between space allocation strategies. AKV (naive) pre-allocates space for two versions, eliminating the need for subsequent space allocation during updates, while AKV allocates space incrementally, initially reserving space for only one version. The experimental results reveal two key findings: (1) AKV (naive) demonstrates superior performance under write-intensive workloads due to its pre-allocation strategy and spatial locality of adjacent versions; (2) AKV outperforms in read-dominated scenarios as it typically requires accessing only a single version, unlike AKV (naive) which necessitates accessing both versions for version determination.

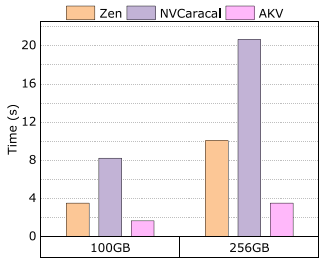


Fig. 17. YCSB recovery.

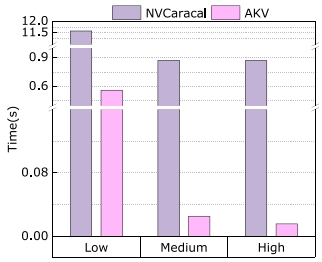


Fig. 18. TPC-C recovery.

E. Recovery Performance

In this section, we evaluate the recovery performance of the evaluated OLTP engines. The capacity ratio of NVM and DRAM is set to 8, with 16 parallel threads for the recovery process. Since the source code of Zen does not provide a complete recovery module, we test its recovery time by measuring its sub-modules, i.e., index construction and tuple cleaning. It should be noted that the actual recovery time for Zen is likely greater than the sum of these measured components. The index construction time measured here is the time required to rebuild the index when running the YCSB test. For tuple cleaning, we refer to the algorithm in [11] to perform a full table scan on the database after the failure, and measured the time required.

Recovery for YCSB: We conduct recovery experiments using YCSB on two databases of different sizes, 100 GB and 256 GB. Before system failure, the storage engine had executed 1.6 million transactions. In Fig. 17, for the 100 GB database, Zen required 3.537 seconds to complete the recovery, of which 1.341 seconds were used to build the index, accounting for about 38% of the recovery time. The recovery time of NVCaracal is mainly spent on tuple scanning and rebuilding indexes, which take 8.174 seconds, while replay transactions take 0.049 seconds. AKV, utilizing an NVM index, spent less than 1 ms for index construction as it only requires address mapping, and 1.678 seconds for recycling of uncommitted transaction modifications. When the database size becomes 256 GB, Zen's total recovery time is 10.059 seconds, an increase of 6.522 seconds compared to the 100 GB database, with 3.537 seconds spent on index construction, approximately 35% of the recovery time. The recovery time of NVCaracal has increased by 12.418 seconds. AKV's recovery time is 3.793 seconds, an increase of 2.115 seconds compared to the 100 GB database. The recovery time of YCSB

will vary with the size of the database, as all three storage engines require a full table scan during fault recovery.

Recovery for TPC-C: In the recovery experiments, the TPC-C database contains 1 (high), 8 (medium), and 256 (low) warehouses, with 16 parallel threads. Before system failure, the storage engine had executed 1.6 million transactions. According to the results in Fig. 18, for 256 warehouses, AKV requires only 0.56 seconds for recovery, of which index reconstruction takes less than 1 ms, and recycling uncommitted transaction modifications takes 0.56 seconds. The recovery time for 8 warehouses is 0.026 seconds. However, For a single warehouse, AKV requires only 0.016 seconds to complete recovery. In comparison, NVCaracal takes about 11.583 seconds to recover 256 warehouses, and both a single warehouse and 8 warehouses take 0.87 seconds to complete. These results clearly show that AKV leverages the non-volatility of NVM to achieve rapid recovery. This is an important discovery because it reveals the superior performance of AKV in failure recovery.

VI. Discussion

In this section, we further discuss several design considerations for AKV. To enhance its versatility, we explore alternative approaches for handling variable-length records. Additionally, to assess potential performance improvements, we consider optional hardware solutions.

Variable-length Record: Our approach primarily focuses on handling fixed-length records, while a straightforward design is employed for variable-length records. This can result in lower tuple space utilization and more frequent space management operations. However, considering the ample size of NVM storage, if certain tuple attributes exceed a predefined length, we can utilize additional space to accommodate them. Specifically, we use pointers to reference an external storage area, thereby maintaining fixed-length records within the tuple heap.

Optional Hardware Solutions: Despite Intel's discontinuation of Optane [41], emerging non-volatile memory technologies like phase-change memory and resistive random access memory retain significant value, with future breakthroughs poised to enhance AKV's architecture. Furthermore, the high-speed interconnect protocol CXL [42], [43], [44], [45] enables persistent memory expansion similar to NVM. These technological advancements indicate that the design principles underlying NVM-based systems are continuously evolving within the emerging hardware ecosystem, providing a broader scope for technological adaptation for the AKV architecture.

VII. Conclusion

This paper presents AKV, a high-throughput, stable, and recoverable OLTP storage engine for NVM. AKV employs an NVM-only architecture, dual-version concurrency control, and circular dual-version storage to simplify system design and enhance performance. Experimental results on YCSB and TPC-C benchmarks demonstrate that AKV achieves higher throughput, lower abort latency, and faster failure recovery than existing engines in most scenarios, with a streamlined architectural footprint. Furthermore, the proposed read abort optimization reduces transaction abort counts by up to 73% for workloads of the same size. AKV provides a viable solution for building OLTP engines on NVM, while optional DRAM acceleration remains available for performance-critical components like indexes.

References

[1] H.-S. P. Wong et al., "Phase change memory," *Proc. IEEE*, vol. 98, no. 12, pp. 2201–2227, Dec. 2010.

[2] S. Ikeda et al., "A perpendicular-anisotropy CoFeB–MgO magnetic tunnel junction," *Nature Mater.*, vol. 9, no. 9, pp. 721–724, 2010.

[3] H.-S. P. Wong et al., "Metal–Oxide RRAM," *Proc. IEEE*, vol. 100, no. 6, pp. 1951–1970, Jun. 2012.

[4] F. Hameed, C. Menard, and J. Castrillon, "Efficient STT-RAM last-level-cache architecture to replace DRAM cache," in *Proc. Int. Symp. Memory Syst.*, 2017, pp. 141–151.

[5] J. A. DeBrabant, J. Arulraj, A. Pavlo, M. Stonebraker, S. B. Zdonik, and S. R. Dulloor, "A prolegomenon on OLTP database systems for non-volatile memory," in *Proc. ADMS, VLDB*, 2014, pp. 57–63.

[6] J. Arulraj, A. Pavlo, and S. R. Dulloor, "Let's talk about storage & recovery methods for non-volatile memory database systems," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, New York, USA, 2015, pp. 707–722.

[7] Y. C. Wang, A. D. Brown, and A. Goel, "Integrating non-volatile main memory in a deterministic database," in *Proc. 18th Eur. Conf. Comput. Syst.*, 2023, pp. 672–686.

[8] A. van Renen et al., "Managing non-volatile memory in database systems," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2018, pp. 1541–1555.

[9] X. Zhou, J. Arulraj, A. Pavlo, and D. Cohen, "Spitfire: A three-tier buffer manager for volatile and non-volatile memory," in *Proc. Int. Conf. Manage. Data*, 2021, pp. 2195–2207.

[10] J. Arulraj, M. Perron, and A. Pavlo, "Write-behind logging," *VLDB Endowment*, vol. 10, no. 4, pp. 337–348, 2016.

[11] G. Liu, L. Chen, and S. Chen, "Zen: A high-throughput log-free OLTP engine for non-volatile main memory," *VLDB Endowment*, vol. 14, no. 5, pp. 835–848, 2021.

[12] Z. Ji, K. Chen, L. Wang, M. Zhang, and Y. Wu, "Falcon: Fast OLTP engine for persistent cache and non-volatile memory," in *Proc. 29th Symp. Operating Syst. Princ.*, New York, USA, 2023, pp. 531–544.

[13] J. Arulraj, J. Levandoski, U. F. Minhas, and P.-A. Larson, "Bztree: A high-performance latch-free range index for non-volatile memory," *VLDB Endowment*, vol. 11, no. 5, pp. 553–565, 2018.

[14] J. Gu et al., "Pisces: A scalable and efficient persistent transactional memory," in *Proc. 2019 USENIX Annu. Tech. Conf.*, 2019, pp. 913–928.

[15] S. Chen et al., "Rethinking database algorithms for phase change memory," *Cidr*, vol. 11, 2011, Art. no. 5th.

[16] S. Chen and Q. Jin, "Persistent B -trees in non-volatile main memory," *Proc. VLDB Endowment*, vol. 8, no. 7, pp. 786–797, 2015.

[17] M. Liu et al., "DudeTM: Building durable transactions with decoupling for persistent memory," *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 329–343, 2017.

[18] F. Xia, D. Jiang, J. Xiong, and N. Sun, "{HiKV}: A hybrid index {Key-Value} store for {DRAM-NVM} memory systems," in *Proc. USENIX Annu. Tech. Conf.*, 2017, pp. 349–362.

[19] Y. Chen, Y. Lu, F. Yang, Q. Wang, Y. Wang, and J. Shu, "Flatstore: An efficient log-structured key-value storage engine for persistent memory," in *Proc. 25th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2020, pp. 1077–1091.

[20] A. N. Udipi, N. Muralimanohar, N. Chatterjee, R. Balasubramonian, A. Davis, and N. P. Jouppi, "Rethinking DRAM design and organization for energy-constrained multi-cores," in *Proc. 37th Annu. Int. Symp. Comput. Archit.*, 2010, pp. 175–186.

[21] L. A. Barroso and U. Hölzle, "The case for energy-proportional computing," *Computer*, vol. 40, no. 12, pp. 33–37, 2007.

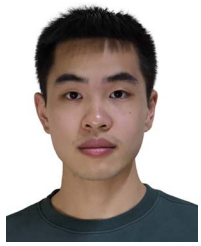
[22] Z. Wang et al., "High-density nand-like spin transfer torque memory with spin orbit torque erase operation," *IEEE Electron Device Lett.*, vol. 39, no. 3, pp. 343–346, Mar. 2018.

[23] Z. Zhang et al., "Plin: A persistent learned index for non-volatile memory with high performance and instant recovery," *Proc. VLDB Endowment*, vol. 16, no. 2, pp. 243–255, 2022.

[24] T. Daulby, A. Savanth, A. S. Weddell, and G. V. Merrett, "Comparing nvm technologies through the lens of intermittent computation," in *Proc. 8th Int. Workshop Energy Harvesting Energy-Neutral Sens. Syst.*, 2020, pp. 77–78.

[25] T. Vogelsang, "Understanding the energy consumption of dynamic random access memories," in *Proc. 43rd Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2010, pp. 363–374.
[26] S. Rai and B. Talawar, "Nonvolatile memory technologies: Characteristics, deployment, and research challenges," *Front. Qual. Electron. Des. AI, IoT Hardware Secur.*, pp. 137–173, 2023.
[27] S. Lee, Y. Ro, Y. H. Son, H. Cho, N. S. Kim, and J. H. Ahn, "Understanding power-performance relationship of energy-efficient modern dram devices," in *Proc. 2017 IEEE Int. Symp. Workload Characterization*, 2017, pp. 110–111.
[28] S. Lee et al., "GreenDIMM: Os-assisted dram power management for dram with a sub-array granularity power-down state," in *Proc. MICRO-54: 54th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2021, pp. 131–142.
[29] A. Thomasian, "Two-phase locking performance and its thrashing behavior," *ACM Trans. Database Syst.*, vol. 18, no. 4, pp. 579–625, 1993.
[30] S. Tu, W. Zheng, E. Kohler, B. Liskov, and S. Madden, "Speedy transactions in multicore in-memory databases," in *Proc. 24th ACM Symp. Operating Syst. Princ.*, 2013, pp. 18–32.
[31] X. Yu, G. Bezerra, A. Pavlo, S. Devadas, and M. Stonebraker, "Staring into the Abyss: An evaluation of concurrency control with one thousand cores," *Proc. VLDB Endowment*, vol. 8, no. 3, pp. 209–220, 2014.
[32] X. Yu, A. Pavlo, D. Sanchez, and S. Devadas, "TicToc: Time traveling optimistic concurrency control," in *Proc. 2016 Int. Conf. Manage. Data*, 2016, pp. 1629–1642.
[33] T. Neumann, T. Mühlbauer, and A. Kemper, "Fast serializable multi-version concurrency control for main-memory database systems," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 677–689.
[34] H. Lim, M. Kaminsky, and D. G. Andersen, "Cicada: Dependably fast multi-core in-memory transactions," in *Proc. ACM Int. Conf. Manage. Data*, 2017, pp. 21–35.
[35] X. Liu, K. Chen, M. Liu, S. Cai, Y. Wu, and W. Zheng, "Multi-clock snapshot isolation concurrency control for NVM database," *Tsinghua Sci. Technol.*, vol. 27, no. 6, pp. 925–938, 2022.
[36] T. Cao et al., "Fast checkpoint recovery algorithms for frequently consistent applications," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2011, pp. 265–276.
[37] H. Mühe, A. Kemper, and T. Neumann, "How to efficiently snapshot transactional data: Hardware or software controlled?," in *Proc. 7th Int. Workshop Data Manage. New Hardware*, 2011, pp. 17–26.
[38] Intel, "Persistent memory development kit." [Online]. Available: https://pmem.io/
[39] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proc. 1st ACM Symp. Cloud Comput.*, New York, NY, USA, 2010, pp. 143–154.
[40] TPC benchmark c. 2020. [Online]. Available: http://www.tpc.org/tpcc/
[41] Intel reports second-quarter 2022 financial results. 2022. [Online]. Available: https://www.intc.com/news-events/press-releases/detail/1563/intel-reports-second-quarter-2022-financial-results
[42] Compute express link. 2022. [Online]. Available: https://www.computeexpresslink.org/
[43] N. Riekenbrauck, M. Weisgut, D. Lindner, and T. Rabl, "A three-tier buffer manager integrating CXL device memory for database systems," in *Proc. IEEE 40th Int. Conf. Data Eng. Workshops*, 2024, pp. 395–401.
[44] M. Jung, "Hello bytes, bye blocks: PCIe storage meets compute express link for memory expansion (CXL-SSD)," in *Proc. 14th ACM Workshop Hot Topics Storage File Syst.*, 2022, pp. 45–51.
[45] D. Koutsoukos, R. Bhartia, M. Friedman, A. Klimovic, and G. Alonso, "NVM: Is it not very meaningful for databases?," *Proc. VLDB Endowment*, vol. 16, no. 10, pp. 2444–2457, 2023.

Jianbin Qin received the BE degree from Northeastern University, China, in 2007, and the PhD degree from the University of New South Wales, Australia, in 2013. He is currently a distinguished professor in the College of Computer Science and Software Engineering, Shenzhen University, and a Research Scientist with the Shenzhen Institute of Computing Sciences. He has won CiSRA best paper award, DASFAA best student paper award, ACM SIGMOD 2011 best paper nomination and invited to contribute ACM TODS, IEEE ICDE 2018 was nominated for best paper and invited to contribute to IEEE TKDE. His research interests include database theory, database systems, similarity query, information retrieval, and data management.



Shuai Liu received the bachelor's degree from Xiangtan University, in 2022 and the master's degree from Shenzhen University, in 2025. His research interests include database systems and NVM.

Tianyu Wang (Member, IEEE) received the PhD degree from the Department of Computer Science and Engineering, The Chinese University of Hong Kong, in 2024. He is currently an assistant professor with the College of Computer Science and Software Engineering, Shenzhen University, China. He has published more than 30 academic papers and has won the SYSTOR 2024 best paper award and the ICESS 2020 best paper award. He also serves as a reviewer for TCAD, TECS, DAC, ICCAD, ICCD, and other journals and conferences. His main research interests include storage system architecture, near/in storage computing systems, non-volatile memory, embedded systems, and blockchain techniques.

Yuxing Chen received the PhD degree in computer science from the University of Helsinki, Finland, in 2021. He currently works as a senior research engineer with the Database R&D Department, Tencent, China. His research interests focus on database performance and evaluation, HTAP database design, and distributed system design.

Anqun Pan is the technical director of the Database R&D Department, Tencent in China. With more than 15 years of experience, he has specialized in the research and development of distributed computing and storage systems. Currently, he is responsible for steering the research and development of the Tencent distributed database system (TDSQL).

Rui Mao received the BS degree and M.S. degrees in computer science from the University of Science and Technology of China, in 1997 and 2000, respectively, the MS degree in statistics, in 2006, and the PhD degree in computer science from the University of Texas at Austin, 2007. He is currently a distinguished professor in the College of Computer Science and Software Engineering, Shenzhen University, and the executive director of Shenzhen Institute of Computing Sciences. His research interests mainly focus on metric-space data processing. His research interests include metric-space data processing, parallel computing, data mining, and machine learning.

Yuxuan Qiu received the PhD degree from the University of Technology Sydney, in 2023. He is currently a professor with the Beijing Institute of Technology, Zhuhai. His research focuses on graph data management and mining, social network analysis, and graph-based machine learning.

Makoto Onizuka received the BE degree from the Tokyo Institute of Technology, in 1991 and the PhD degree from Tokyo Institute of Technology, in 2007. He is a professor with the Graduate School of Information Science and Technology, the University of Osaka. He is engaged in research on a technical field of distributed/parallel data analysis, efficient graph analysis algorithms, and exploratory data mining.

Chuan Xiao received the BE degree from Northeastern University, China, in 2005 and the PhD degree from the University of New SouthWales, in 2011. He is an associate professor with the Institute for Advanced Co-Creation Studies, the University of Osaka, and the School of Informatics, Nagoya University. His research interests include computer simulation, data preprocessing, data integration, and natural language processing.